

detecting outliers:

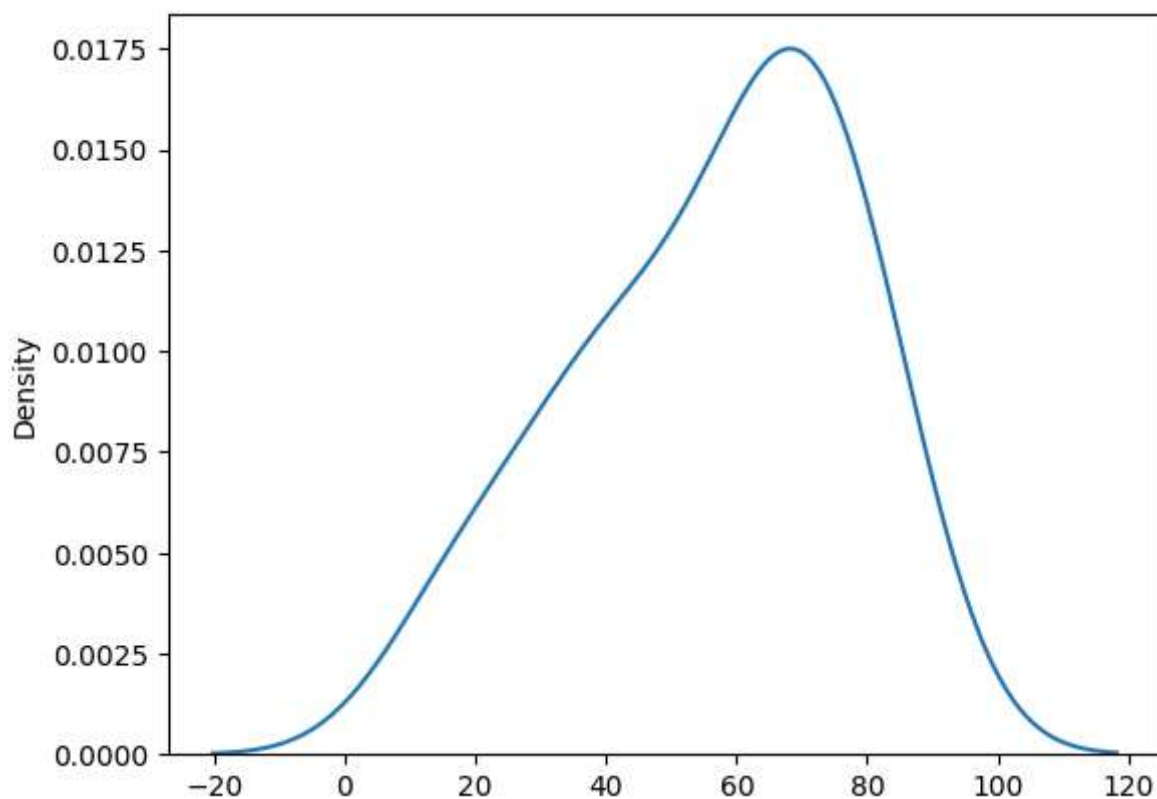
In [238... `import numpy as np`

In [239... `array=np.random.randint(10,100,size=20)`
`array`

Out[239... `array([36, 62, 81, 41, 52, 55, 47, 84, 62, 80, 74, 69, 68, 14, 20, 68, 77,
67, 29, 39], dtype=int32)`

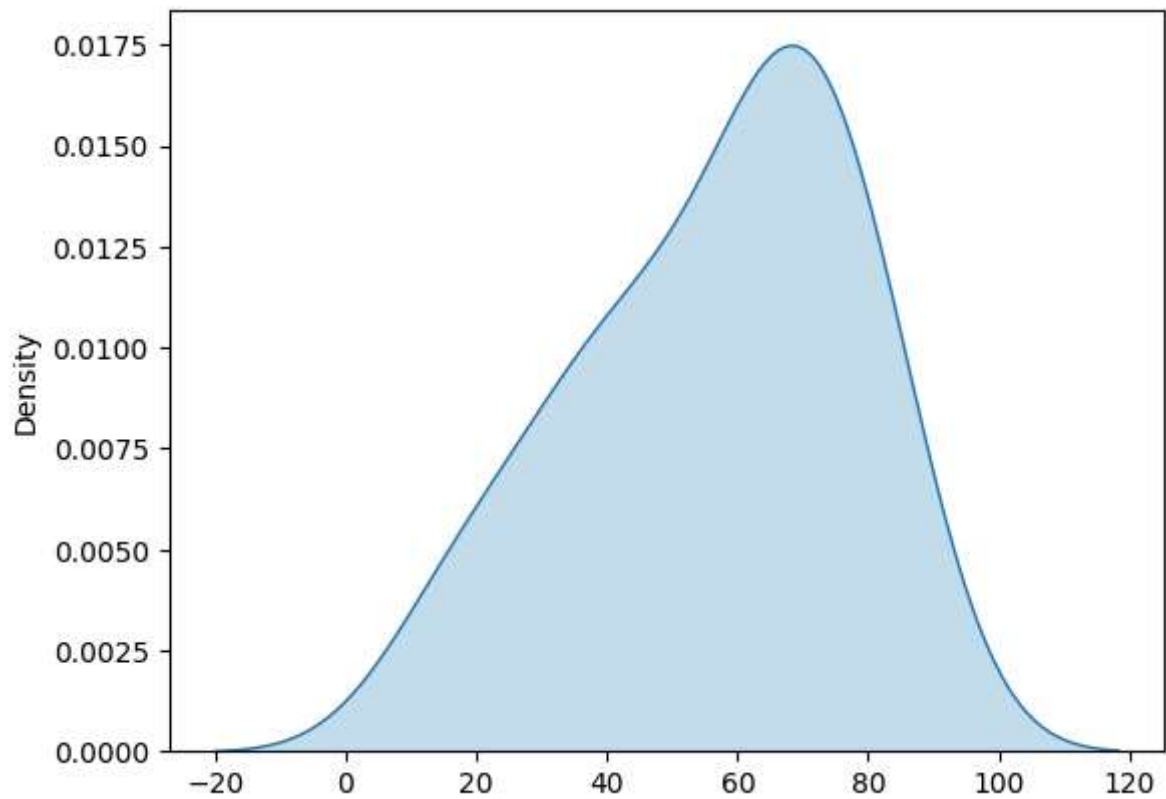
In [240... `import seaborn as sns`
`sns.kdeplot(array)`

Out[240... `<Axes: ylabel='Density'>`



In [241... `import seaborn as sns`
`sns.kdeplot(array,fill=True)`

Out[241... `<Axes: ylabel='Density'>`

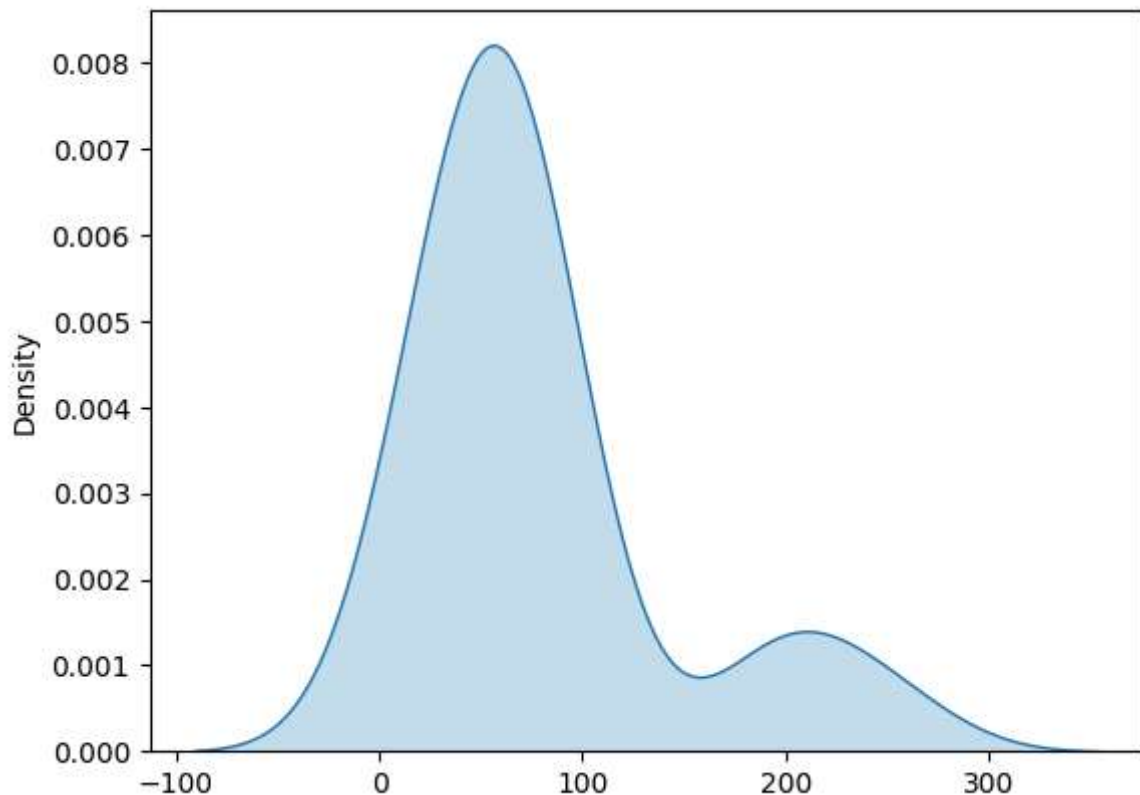


```
In [242... array2=array.copy()  
array2[10]=200  
array2[12]=250  
array2[5]=200  
array2
```

```
Out[242... array([ 36,  62,  81,  41,  52, 200,  47,  84,  62,  80, 200,  69, 250,  
          14,  20,  68,  77,  67,  29,  39], dtype=int32)
```

```
In [243... sns.kdeplot(array2,fill=True)
```

```
Out[243... <Axes: ylabel='Density'>
```



Z-score normalization

$z_score = (x_value - x_mean) / std$

calculate z_score and setting up a threshold such as above or below threshold are outliers which can be eliminated.

In [247...

```
mean=np.mean(array2)
std=np.std(array2)
print('mean is ',mean,'std is',std)
```

mean is 78.9 std is 61.7866490433006

In [248...

```
outliers_list=[]
threshold=1.5

for x_val in array2:
    z_score=(x_val-mean)/std
    z_score=np.abs(z_score)
    print(z_score)

    if z_score >= threshold:
        outliers_list.append(x_val)

outliers_list
```

```

0.6943247556593549
0.27352187344156415
0.03398792510220609
0.6134011244636259
0.4353691358330221
1.9599703475605563
0.5162927670287512
0.0825421038196435
0.27352187344156415
0.01780319886306029
1.9599703475605563
0.1602287897675435
2.7692066595178466
1.0503887329205626
0.9532803754856878
0.1764135160066893
0.030750979854377112
0.19259824224583513
0.8076178393333755
0.6457705769419175

```

```
Out[248...] [np.int32(200), np.int32(200), np.int32(250)]
```

```
In [251...] np.where(array2<+min(outliers_list),np.mean(array2),array2)
```

```
Out[251...] array([ 78.9,  78.9,  78.9,  78.9,  78.9, 200. ,  78.9,  78.9,  78.9,
        78.9, 200. ,  78.9, 250. ,  78.9,  78.9,  78.9,  78.9,  78.9,
        78.9,  78.9])
```

using IQR(inter quartile range):

```
In [252...] array
```

```
Out[252...] array([36, 62, 81, 41, 52, 55, 47, 84, 62, 80, 74, 69, 68, 14, 20, 68, 77,
        67, 29, 39], dtype=int32)
```

```
In [254...] q1=np.quantile(array,0.25)
q1
```

```
Out[254...] np.float64(40.5)
```

```
In [255...] q2=np.quantile(array,0.5)
q2
```

```
Out[255...] np.float64(62.0)
```

```
In [258...] q3=np.quantile(array,0.75)
q3
```

```
Out[258...] np.float64(70.25)
```

```
In [260...] import pandas as pd
pd.Series(array).describe()
```

```
Out[260... count    20.000000
          mean    56.250000
          std     20.774162
          min     14.000000
          25%     40.500000
          50%     62.000000
          75%     70.250000
          max     84.000000
          dtype: float64
```

IQR

Q1 >> 1st quantile >> 25th percentile of the data Q2 >> 2nd quantile >> 50th percentile of the data/median Q3 >> 3rd quantile >> 75th percentile of the data

$IQR = Q3 - Q1$

$lower_point = Q1 - 1.5 \cdot IQR$ $upper_point = Q3 + 1.5 \cdot IQR$

```
In [261... df=pd.read_csv(r"C:\Users\Lenovo\Documents\Notes\MARCH\18th - ML\TITANIC PROJECT\ti
```

```
In [262... df
```

Out[262...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05
...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.73

891 rows × 12 columns



```
In [263... df.dropna(subset=['Age'],inplace=True)
```

```
In [264... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 714 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  714 non-null    int64
1   Survived     714 non-null    int64
2   Pclass       714 non-null    int64
3   Name         714 non-null    object
4   Sex          714 non-null    object
5   Age          714 non-null    float64
6   SibSp        714 non-null    int64
7   Parch        714 non-null    int64
8   Ticket       714 non-null    object
9   Fare         714 non-null    float64
10  Cabin        185 non-null    object
11  Embarked     712 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 72.5+ KB
```

```
In [265... df.Age.describe()
```

```
Out[265... count    714.000000
mean      29.699118
std       14.526497
min        0.420000
25%       20.125000
50%       28.000000
75%       38.000000
max       80.000000
Name: Age, dtype: float64
```

```
In [266... import warnings
warnings.filterwarnings("ignore")
```

```
In [267... Q1=df.Age.describe()[4]
Q2=df.Age.describe()[5]
Q3=df.Age.describe()[6]
```

```
In [268... print(Q1,Q2,Q3)
```

```
20.125 28.0 38.0
```

```
In [271... Q1=df.Age.quantile(0.25)
Q2=df.Age.quantile(0.5)
Q3=df.Age.quantile(0.75)
```

```
In [272... print(Q1,Q2,Q3)
```

```
20.125 28.0 38.0
```

```
In [273... IQR = Q3-Q1
print(IQR)

lower_bound =Q1 - 1.5*IQR
upper_bound= Q3 + 1.5*IQR

print(lower_bound,upper_bound)
```

```
17.875
-6.6875 64.8125
```

```
In [275... df.loc[(df.Age>upper_bound)| (df.Age<lower_bound)][ 'Age' ]
```

```
Out[275... 33      66.0
54      65.0
96      71.0
116     70.5
280     65.0
456     65.0
493     71.0
630     80.0
672     70.0
745     70.0
851     74.0
Name: Age, dtype: float64
```

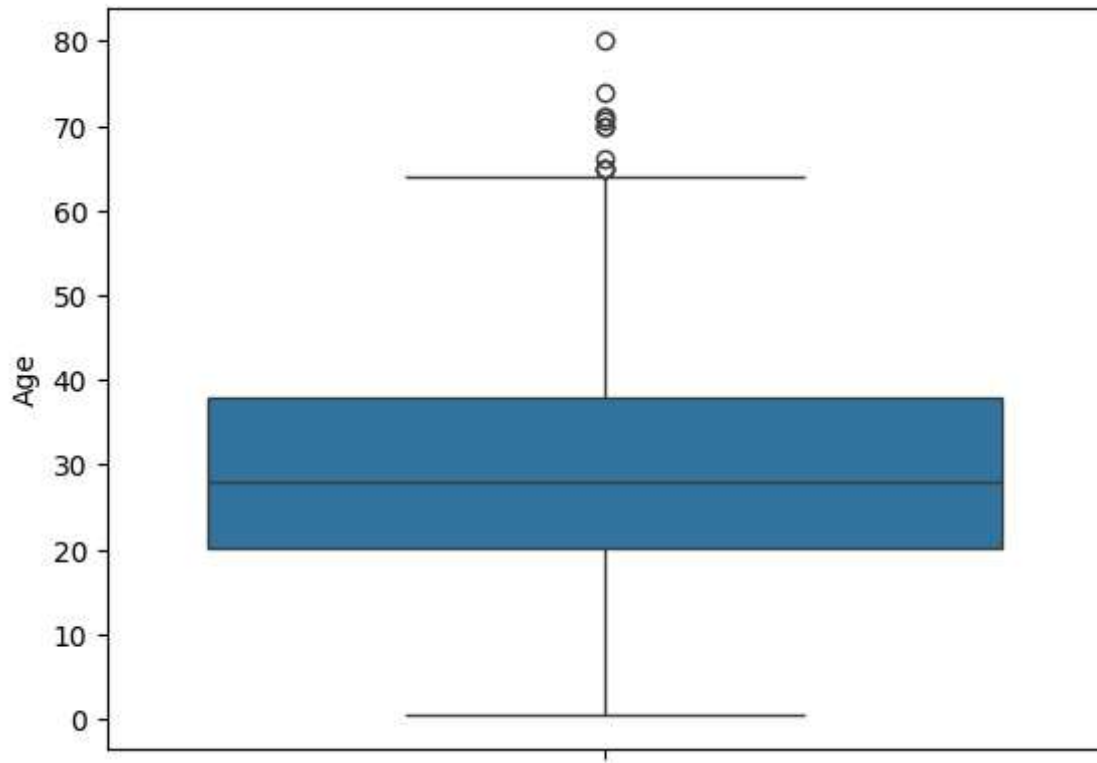
```
In [279... outlier_indices=df.loc[(df.Age>upper_bound)|(df.Age<lower_bound)].index
outlier_indices
```

```
Out[279... Index([33, 54, 96, 116, 280, 456, 493, 630, 672, 745, 851], dtype='int64')
```

detecting outlier using boxplot

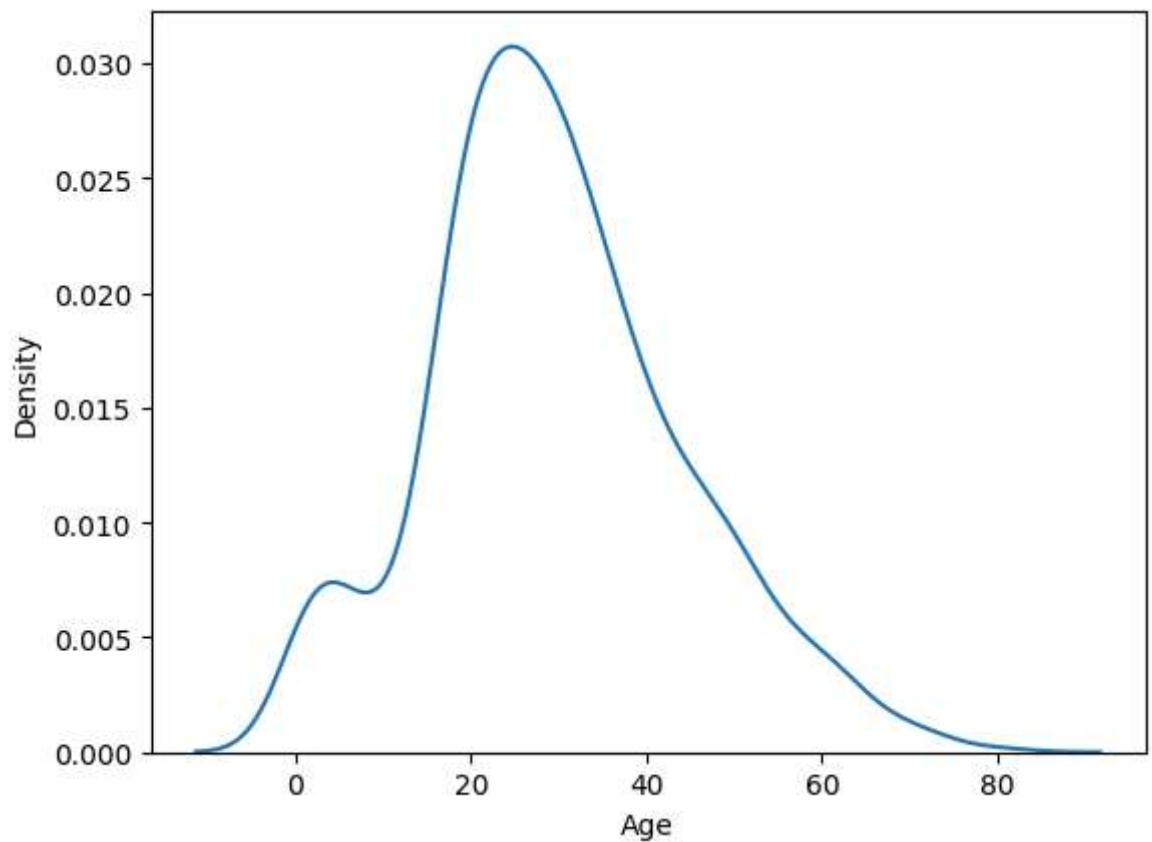
```
In [280... sns.boxplot(df.Age)
```

```
Out[280... <Axes: ylabel='Age'>
```

```
In [281... sns.kdeplot(df.Age)
```

```
Out[281... <Axes: xlabel='Age', ylabel='Density'>
```



droppping outliers:

```
In [283... df.loc[-((df.Age>upper_bound)|(df.Age<lower_bound))]
```

Out[283...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05
...	...	...	...	...	...	...	...	...	...	...
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.12
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75

703 rows × 12 columns



imputing for outliers:

```
In [284... median =df.Age.median()
median
```

```
Out[284... np.float64(28.0)
```

```
In [286... mean_age = df.loc[(df.Age<upper_bound)&(df.Age>10)]['Age'].mean()
print('mean age prior to removing outliers is',mean_age)
```

```
df.Age.loc[df.Age>upper_bound] =mean_age
print('mean age after imputing for outliers is',df.Age.mean())
```

mean age prior to removing outliers is 31.556338028169016

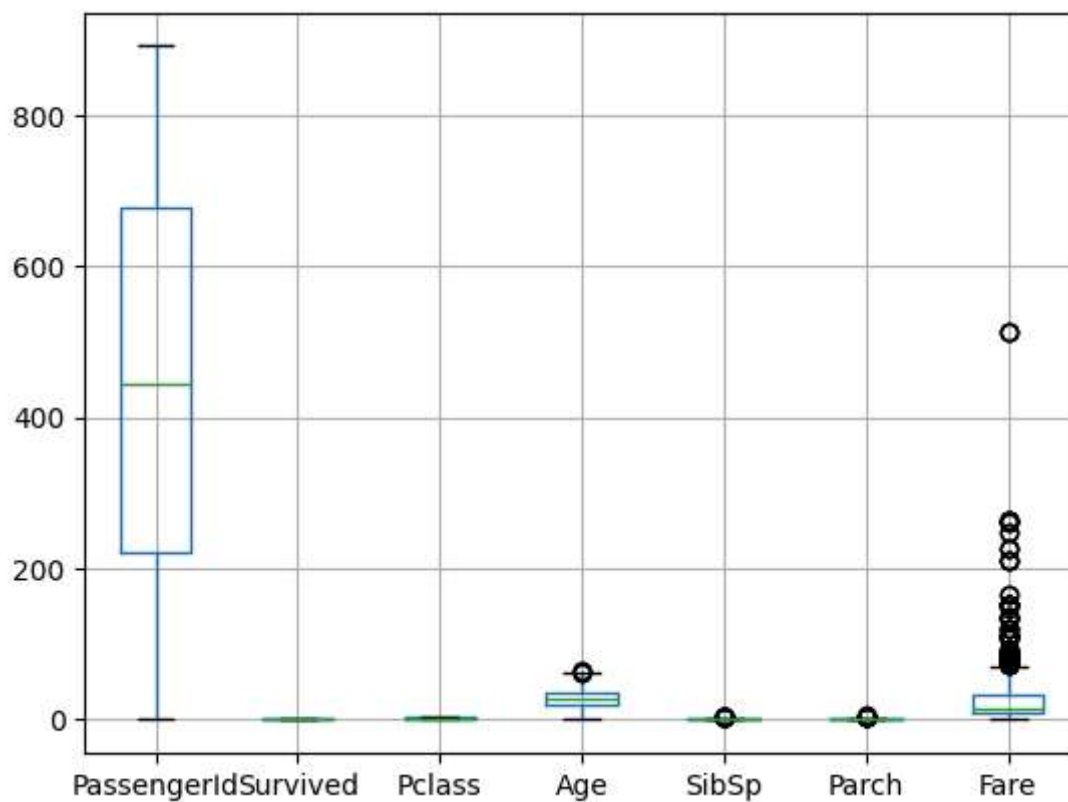
mean age after imputing for outliers is 29.11034974553202

```
In [288... df.Age.loc[df.Age>upper_bound]
```

```
Out[288... Series([], Name: Age, dtype: float64)
```

```
In [290... df.boxplot()
```

```
Out[290... <Axes: >
```



```
In [291... df.boxplot(["Age", "Fare"])
```

```
Out[291... <Axes: >
```

